

CS-1004: Object Oriented Programming (CS)

Serial No:

Sessional Exam-II**Total Time: 1 Hour****Total Marks: 60**Monday, 10th April, 2023**Course Instructors**

Amna Irum, Shams Farooq, Shehreyar Rashid,
Mariam Hida

Signature of Invigilator

Anoosha Ahmed M. Lk 201-1167 CS-B
Student Name Roll No. Section

Signature

DO NOT OPEN THE QUESTION BOOK OR START UNTIL INSTRUCTED.

Instructions:

1. Attempt on question paper. Attempt all of them. Read the question carefully, understand the question, and then attempt it.
2. No additional sheet will be provided for rough work. Use the back of the last page for rough work.
3. If you need more space write on the back side of the paper and clearly mark question and part number etc.
4. After asked to commence the exam, please verify that you have **ten (10)** different printed pages including this title page. There are a total of **2** questions. And write your instructor's complete office number in front of point 6 to secure bonus marks.
5. Calculator sharing is strictly prohibited.
6. Use permanent ink pens only. Any part done using soft pencil will not be marked and cannot be claimed for rechecking.

C-205F

	Q-1	Q-2	Total
Marks Obtained	30	20	50
Total Marks	30	30	60

V. good

+ 5 = 55

Question 01 [30 marks]

What would be the output produced by executing the following C++ codes? Identify errors, if any (either write output or error (syntax/runtime), both will not be accepted). All the code snippet contains `#include<iostream>` and using namespace std;

A [3 Marks]

```
class Box{
    int capacity;
    bool operator<(Box b){
        return this->capa city < b.capacity ? true : false;}
public:
    Box(){}
    Box(double capacity){ this->capacity = capacity; }
};
int main(){
    Box b1(10);
    Box b2 (14);
    if(b1 < b2){
        cout<<"Box 2 has large capacity.";
    }
    else{
        cout<<"Box 1 has large capacity.";
    }
    return 0;
}
```

Output/Error:

Error
bcz overloaded(<) is private, so not accessible inside main().

B [5 Marks]

```
class Point{
    int x, y;
public:
    Point(int x=0, int y=0){
        this->x=x;    Point::y=y;
        (*this)();
    }
    void operator()(){
        cout<<" ("<<x<<","<<y<<") " <<endl;
    }
    Point& operator()(int y){
        this->y=y;
        return *this;
    }
    ~Point(){
        cout<<"Point is going"; (*this)();
    }
} p3;
int main() {
    ✓Point *p=new Point(5,6);
    static Point p1(p3);
    ✓p1(9)(8);
    ✓delete p;
    Point p2(7);
    cout<<"-----" <<endl;
    return 0;
}
```

Output/Error:

~~(0,0)~~
(0,0) ✓
(5,6) ✓
Point is going (5,6) ✓
(7,0) ✓
----- ✓
Point is going (7,0) ✓
Point is going (0,8) ✓
Point is going (0,0) ✓

National University of Computer and Emerging Sciences

FAST School of Computing

Spring-2023

Islamabad Campus

C [7 Marks]

```

class Complex{
    double r, i;
public:
    Complex(double r=1.0,double i=1.0){
        set(r,i);
    }
    void set(double r,double i){
        Complex::r = r; this->i = i;
    }
    void print(){
        if (i<0)
            cout << r << "" << i << "i" << endl;
        else
            cout << r << "+" << i << "i" << endl;
    }
    Complex operator+(Complex R){
        Complex tmp;
        tmp.r = r + R.r;
        tmp.i = i + R.i;
        return tmp;
    }
    Complex operator++(){
        Complex tmp=*this;
        r++; i++;
        return tmp;
    }
    Complex operator++(int){
        ++(*this);
        return *this;
    }
};

int main()
{
    ✓ Complex A(3,4), B(5,-6);
    ✓ A.print();
    ✓ B.print();
    ✓ Complex C;
    ✓ C = A+B;
    ✓ C.print();
    ✓ (++A).print();
    ✓ C = ++A;
    ✓ C.print();
    ✓ (A++).print();
    ✓ A.print();
    return 0;
}

```

post fixed

pre fixed

Output/Error:

$3 + 4i$ ✓
 $5 - 6i$ ✓
 $8 - 2i$ ✓
 $3 + 4i$ ✓
 $4 + 8i$ ✓
 $6 + 7i$ ✓
 $6 + 7i$ ✓

7

D [5 Marks]

```
class Mystery {  
public:  
static int n;  
Mystery() {cout << n++<<endl; }  
Mystery(int i) {n=i;cout << n<<endl;}  
static void somefunc(){ n=5;}  
  
Mystery(Mystery const& otherNum){  
    n+=5;  
    cout << n<<endl;  
}  
~Mystery(){cout<< --n <<" ";}  
}a;  
  
void fun(Mystery n){  
    cout<<n.n<<endl;  
    n.somefunc();  
}  
int Mystery::n=0;  
  
int main(){  
    Mystery b(9), c;  
    fun(b);  
    return 0;  
}
```

Output/Error:

0 ✓

9 ✓

9 ✓

15 ✓

15 ✓

4 3 2 1

✓

5

National University of Computer and Emerging Sciences

FAST School of Computing

Spring-2023

Islamabad Campus

E [10 Marks]

```
class ZooCage
{
    int cageNumber;
    ZooCage* link;
public:
    ZooCage(int n):cageNumber(n),link(NULL){}
    int getCageNumber() {return cageNumber;}
    ZooCage*& getLink() {return link;}
} *start=NULL;
void fun(ZooCage *& H, int num)
{
    if(H){
        fun(H->getLink(),num);
        return;
    }
    H = new ZooCage(num);
}
void fun(ZooCage * H)
{
    if(H){
        fun(H->getLink());
        cout<<H->getCageNumber()<<endl;
    }
}
int main(){
    fun(start,4);
    fun(start,2);
    ZooCage * temp=new ZooCage(5);
    temp->getLink()=start->getLink();
    start->getLink()=temp;
    fun(start,3);
    fun(start);
    return 0;
}
```

Output/Error:

~~3~~

3 ✓

2 ✓

5 ✓

4 ✓

10

Question 02 [30 marks]

Your goal is to write a program for creating a MovieStore. Your MovieStore should allow for storage of many movie names, there (unique) IDs and ratings. Your MovieStore should allow facility of performing following operations, (minimum) data members required are given (you can add more). You need to add the necessary functions as per sequence, following the sequence will give you 2 marks:

```
int main(){
    MovieStore s1, s2(10); // create two stores of size 5 and 10

    s1["Hobbit"] = 4.5;
    /* If previously not added, add a movie with average rating of 4.5
    If already exist, overwrite rating by 4.5
    If memory finished print "Memory finished overwriting rating of last movie"
    And Update accordingly */

    s1["Lord of the Rings"] = 5;

    cout << s1[2] << endl;
    /* get rating for movie id 2 (Lord of the Rings => 5).
    If id not found print "id not found"
    And return -1 rating. */

    s2=s1; // copies elements from s1 to s2

    s1["Lion King"] = 8;
    cout << s2; // should display all the Movie store information

    return 0;
}
```

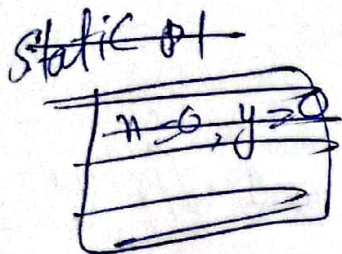
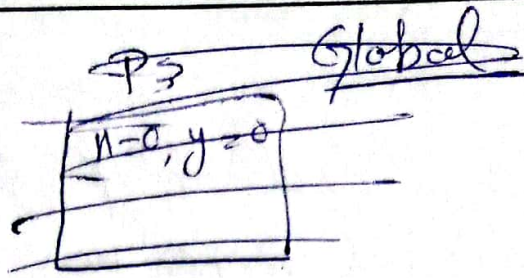
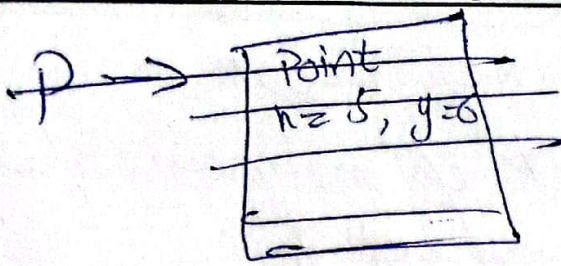
Expected Output:

5

MovieStore:

Id	Movie	Rating
1	Hobbit	4.5
2	Lord of the Rings	5

```
class MovieStore{
    string * names;
    int * ids;
    float * rating;
    int size;
    int currentSize;
    .....
```

D'

public:

MovieStore (int size=5)

```
{ this->size = size; this->currentSize = 0;
```

```
this->names = new string[size];
```

```
this->ids = new int[size];
```

```
this->rating = new float[size];
```

}

```
float & operator[] (string name)
```

```
{ // bool flag = false;
```

```
for (int i=0; i<this->currentSize; i++)
```

```
{ if (this->names[i] == name)
```

```
{ return this->rating[i];
```

}

```
this->names[currentSize] = name;
```

```
if (this->currentSize == this->size - 1)
```

```
{ cout << "Memory finished overwriting  
rating of last movie";
```

```
this->names[currentSize] = name;
```



```

return this->rating[currentSize];
} // End of if to check if memory is full
// Control reaches here if memory not full
this->names[currentSize] = name;
this->currentSize++;
return this->rating[currentSize]; // return currentSize-1
} // End of function

```

```

float operator[] (float int number)
{
    if (this->currentSize + 1 == number)
    {
        cout << "id not found";
        return -1;
    }
    // if id exceeds currentSize
    return this->rating[number - 1];
}

```

```

MovieStore (const &MovieStore M)
{
    this->size = M.size;
    this->currentSize = M.currentSize;
    for (int i = 0; i <
    this->names = new string[this->size];
    this->ids = new int[this->size];
}

```



```

this->rating = new float[this->size];
for (int i = 0; i < this->currentSize; i++)
{
    this->names[i] = M.names[i];
    this->rating[i] = M.rating[i];
    this->ids[i] = M.ids[i];
}

```

} // End of function

~ MovieStore ()

```

{
    delete [] names;
    delete [] ids;
    delete [] ratings;
}

```

3

MovieStore operator = (const &MovieStore M)

```

{
    this->size = M.size; this->currentSize = M.currentSize;

```

```

    this->names = new string[size];

```

```

    this->ids = new int[size];

```

```

    this->rating = new float[size];

```

```

    for (int i = 0; i < currentSize; i++)
    {

```

```

        this->names[i] = M.names[i];

```

```

        this->rating[i] = M.rating[i];

```

```

        this->ids[i] = M.ids[i];
    }
}

```